2026-10-25 09:28:38 You

🔴 Level 9 — Advanced Ansible

You don't need to spend huge amounts of time here initially.

84. **Blocks**
85. **Rescue**
86. **Always**
87. **Error handling**
88. **Delegation**
89. **`delegate_to`**
90. **`run_once`**
91. **Local actions**
92. **Serial execution**
93. **Rolling deployments**
94. **Async tasks**
95. **Retries / Until**
96. **Wait for**
97. **Conditionals**
98. **Complex loops**
99. **Custom modules**
100. **Ansible configuration (`ansible.cfg`)**

ChatGPT gpt-6

Ansible Level 9 — Advanced Ansible

Goal: Understand the advanced features used to handle failures, coordinate multiple servers, deploy applications safely, and control Ansible execution.

You don't need to master everything immediately. Focus first on **Blocks, error handling, delegation, rolling deployments, retries, and** `ansible.cfg`.

84. Blocks

Purpose: Group multiple tasks together so you can apply shared settings, conditions, and error handling.

Example:

```yaml
- name: Configure web server
  hosts: web
  become: true

  tasks:
    - name: Configure Nginx
      block:
        - name: Install Nginx
          ansible.builtin.apt:
            name: nginx
            state: present

        - name: Start Nginx
          ansible.builtin.service:
            name: nginx
            state: started
```

Remember:

- `block` groups related tasks.
- You can apply `when` and `become` at the block level.
- `block` can have `rescue` and `always` sections.

Production example: Group application installation, configuration, and startup tasks into one logical unit.

---

85. Rescue

Purpose: Run recovery tasks when a task inside a block fails.

Example:

```yaml
- name: Deploy application
  block:
    - name: Run deployment script
      ansible.builtin.command: /opt/myapp/deploy.sh

  rescue:
    - name: Report deployment failure
      ansible.builtin.debug:
        msg: "Deployment failed. Investigate the issue."
```

How it works:

1. Ansible runs the deployment script.
2. If the task fails, Ansible runs the `rescue` tasks.
3. If rescue succeeds, Ansible generally treats the original block failure as recovered for control-flow purposes.

Production example: If a deployment step fails, run diagnostic commands or attempt a rollback.

Important: A successful rescue does not automatically restore the application to its previous state. You must implement the recovery or rollback logic yourself.

---

86. Always

Purpose: Run cleanup or reporting tasks after a block completes, whether its tasks succeed or fail.

Example:

```yaml
- name: Deployment workflow
  block:
    - name: Run deployment
      ansible.builtin.command: /opt/myapp/deploy.sh

  rescue:
    - name: Report failure
      ansible.builtin.debug:
        msg: "Deployment failed"

  always:
    - name: Record workflow completion
      ansible.builtin.debug:
        msg: "Deployment workflow finished"
```

Remember:

Section	Purpose
`block`	Main tasks
`rescue`	Handle failures

Section	Purpose
`always`	Cleanup or reporting

Production example: Remove temporary files, release a maintenance lock, or record deployment status.

`always` runs after the block's normal or rescue path, but it is not a guarantee against controller crashes, unreachable hosts, or other interruptions that prevent execution.

---

87. Error Handling

Purpose: Control what Ansible does when a task fails.

By default, a failed task stops further execution for that host.

A. `ignore_errors`

Continue to the next task if a task fails.

```yaml
</> YAML

- name: Check optional service
  ansible.builtin.command: systemctl status optional-app
  ignore_errors: true
```

Use this only when the failure is acceptable and understood.

B. `failed_when`

Define what should count as a failure.

```yaml
</> YAML

- name: Check application response
  ansible.builtin.command: /opt/myapp/health-check.sh
  register: health
  failed_when: "'UNHEALTHY' in health.stdout"
```

The task fails if the output contains `UNHEALTHY`.

C. `changed_when`

Control whether Ansible reports a task as changed.

```yaml
- name: Check application version
  ansible.builtin.command: /opt/myapp/version.sh
  changed_when: false
```

Remember:

- `ignore_errors` → Continue after certain task failures.
- `failed_when` → Define failure conditions.
- `changed_when` → Define change reporting.

Production example: Run a health-check script and explicitly fail deployment if it reports an unhealthy application.

---

88. Delegation

Purpose: Execute a task on a different host from the one currently being processed.

Example scenario: You are configuring three web servers, but you need to run a load-balancer API call on a separate control host.

```yaml
- name: Configure web servers
  hosts: web
  tasks:
    - name: Run command on deployment controller
      ansible.builtin.command: hostname
      delegate_to: localhost
```

Although Ansible processes each web host, the command runs on `localhost`.

Production example: Remove a web server from the load balancer before deploying it.

Remember: Delegation changes where the task executes. It does not change the play's original host selection.

---

89. `delegate_to`

Purpose: Specify the host on which a particular task should execute.

It is the task keyword that implements delegation.

Example — Run a load-balancer command on a specific host

```yaml
- name: Deploy web application
  hosts: web
  tasks:
    - name: Disable server in load balancer
      ansible.builtin.command:
        argv:
          - /usr/local/bin/lbctl
          - disable
          - "{{ inventory_hostname }}"
      delegate_to: lb01
```

If the current host is `web01`, the command executes on `lb01` and passes `web01` as the server to disable.

Important:

- `inventory_hostname` identifies the original host being processed.
- `delegate_to` identifies where the delegated task executes.
- The delegated host must be reachable and have the necessary tools and permissions.

Production example: Run a database registration command on a database administration host while processing application servers.

---

90. `run_once`

Purpose: Execute a task only once for the current batch of hosts instead of once per host.

Example

```yaml
- name: Deploy application
  hosts: web
  tasks:
    - name: Display deployment start
      ansible.builtin.debug:
        msg: "Starting deployment"
      run_once: true
```

Without `run_once`, the task normally runs for each host. With it, the task runs once for the batch.

Production example: Run a shared database migration once rather than once on every web server.

Important: `run_once` is not a global exactly-once guarantee. With `serial`, it can run once per batch. For operations that must never run concurrently or twice, use appropriate locking or coordination.

---

91. Local Actions

Purpose: Execute a task on the Ansible control node instead of a remote target.

The modern equivalent is usually `delegate_to: localhost`.

Example:

```yaml
- name: Deploy application
  hosts: web
  tasks:
    - name: Create local deployment report
      ansible.builtin.copy:
        content: "Deployment started"
        dest: /tmp/deployment-report.txt
      delegate_to: localhost
      run_once: true
```

This writes the report on the control node.

Ansible also supports the shorthand `local_action`:

```yaml
- name: Run local command
  hosts: web
  tasks:
    - name: Check local Ansible version
      ansible.builtin.command: ansible --version
      delegate_to: localhost
      run_once: true
```

Production example: Call a deployment API or generate a report on the controller while configuring remote servers.

Remember: `local_action` is shorthand for a delegated task. Prefer `delegate_to: localhost` when you want the execution location to be obvious.

---

92. Serial Execution

Purpose: Control how many hosts Ansible processes at a time.

By default, Ansible can process multiple hosts concurrently, subject to the configured forks and other execution limits.

Example:

```yaml
</> YAML

- name: Deploy application
  hosts: web
  serial: 1
  tasks:
    - name: Update application
      ansible.builtin.command: /opt/myapp/deploy.sh
```

With `serial: 1`, Ansible processes one host at a time.

Other examples:

```yaml
</> YAML

serial: 3
```

Process hosts in batches of three.

```yaml
</> YAML

serial: "25%"
```

Process approximately 25% of the selected hosts per batch, subject to Ansible's batch sizing rules.

Production example: Update one web server at a time to reduce the risk of taking all application instances offline.

Remember: `serial` controls batch size; `forks` controls Ansible's overall parallelism. They serve different purposes.

---

93. Rolling Deployments

Purpose: Update application instances gradually rather than deploying to every server simultaneously.

A rolling deployment combines techniques such as:

- `serial` to control batch size.
- Delegation to manage load-balancer membership.
- Health checks to verify application readiness.
- Handlers to restart services when required.
- Rescue or rollback logic to handle failures.

Basic example

```yaml
- name: Rolling application deployment
  hosts: web
  serial: 1
  become: true

  tasks:
    - name: Deploy new application version
      ansible.builtin.command: /opt/myapp/deploy.sh

    - name: Verify application health
      ansible.builtin.command: /opt/myapp/health-check.sh
      changed_when: false
```

This deploys and checks each host in sequence.

Production workflow

1. Remove web01 from load balancer

2. Deploy new version to web01

3. Run health check

4. Re-add web01, then proceed to web02

Important: The basic playbook above does not actually remove hosts from a load balancer or re-add them. Those actions must be implemented using your load balancer's API, module, or command.

Production example: Deploy an application across 10 web servers one at a time, validating each instance before proceeding.

---

94. Async Tasks

Purpose: Start a long-running task without making Ansible wait for it to finish immediately.

Use `async` and `poll`.

Example:

```yaml
- name: Run long application upgrade
  ansible.builtin.command: /opt/myapp/upgrade.sh
  async: 1800
  poll: 10
```

Meaning:

- `async: 1800` → Set the maximum allowed runtime to 1,800 seconds.
- `poll: 10` → Check for completion every 10 seconds.

Ansible waits for the task while polling periodically, rather than waiting in one uninterrupted execution.

Fire and continue

```yaml
- name: Start long backup
  ansible.builtin.command: /usr/local/bin/backup.sh
  async: 3600
  poll: 0
  register: backup_job
```

With `poll: 0`, Ansible starts the asynchronous job and moves on without waiting for completion.

You can check the job later:

```yaml
- name: Wait for backup completion
  ansible.builtin.async_status:
    jid: "{{ backup_job.ansible_job_id }}"
  register: backup_status
  until: backup_status.finished
  retries: 60
  delay: 10
```

Production example: Start a long database backup or software installation while allowing other automation tasks to continue.

Remember: `async` does not automatically make a job safe to repeat. Consider timeouts, cleanup, and recovery if the task fails.

---

95. Retries / Until

Purpose: Retry a task until a condition becomes true or the retry limit is reached.

Useful when an application takes time to start or a service is temporarily unavailable.

Example — Retry a health check

```yaml
</> YAML

- name: Wait for application readiness
  ansible.builtin.command: /opt/myapp/health-check.sh
  register: health_result
  until: health_result.rc == 0
  retries: 10
  delay: 5
  changed_when: false
```

How it works:

1. Run the health-check script.
2. If its exit code is `0`, stop retrying.
3. Otherwise, retry up to the configured limit.
4. Wait five seconds between attempts.

Important parameters

Parameter	Purpose
`register`	Stores the task result
`until`	Condition that must become true
`retries`	Maximum retry attempts
`delay`	Seconds between attempts

Production example: Wait for an application to become healthy after restarting it.

Remember: `until` checks a condition after each attempt. It is not the same as `ignore_errors`, which allows execution to continue despite a failure.

---

96. Wait For

Purpose: Wait for a port, file, or other supported condition to become available.

Module: `ansible.builtin.wait_for`

Example — Wait until an application port is open

```yaml
- name: Wait for application
  ansible.builtin.wait_for:
    host: 127.0.0.1
    port: 8080
    state: started
    timeout: 120
```

Ansible waits up to 120 seconds for port `8080` to accept connections.

Wait for a file to exist

```yaml
- name: Wait for deployment completion marker
  ansible.builtin.wait_for:
    path: /opt/myapp/deployment.complete
    state: present
    timeout: 300
```

Common parameters

Parameter	Meaning
`host`	Host to check
`port`	Port to monitor
`path`	File path to monitor
`state: started`	Wait for a port to become available

Parameter	Meaning
`state: stopped`	Wait for a port to become unavailable
`timeout`	Maximum waiting time

Production example: Wait for a web application's port to open before performing a health check.

Remember: An open port proves connectivity, not necessarily application correctness. Follow it with a health endpoint or application-specific check when needed.

---

97. Conditionals

Purpose: Run tasks only when specified conditions are met.

The main keyword is `when`.

A. Check a variable

```yaml
- name: Install Nginx in production
  ansible.builtin.apt:
    name: nginx
    state: present
  when: environment == "production"
```

B. Check the operating system

```yaml
- name: Install Nginx on Ubuntu
  ansible.builtin.apt:
    name: nginx
    state: present
  when: ansible_facts['distribution'] == "Ubuntu"
```

C. Check a previous task result

```yaml
- name: Check application
  ansible.builtin.command: /opt/myapp/health-check.sh
```

```yaml
  register: health_result
  changed_when: false
  failed_when: false

- name: Display warning if unhealthy
  ansible.builtin.debug:
    msg: "Application health check failed"
  when: health_result.rc != 0
```

Production example: Install packages according to the operating system, deploy environment-specific settings, or trigger recovery only when a health check fails.

Remember: `when` expects a condition, not a Jinja2 expression wrapped in `{{ }}`.

Correct:

```yaml
when: environment == "production"
```

Avoid:

```yaml
when: "{{ environment == 'production' }}"
```

---

98. Complex Loops

Purpose: Repeat tasks over lists, dictionaries, or structured data.

The modern preferred keyword is `loop`.

A. Loop through package names

```yaml
- name: Install required packages
  ansible.builtin.apt:
    name: "{{ item }}"
    state: present
  loop:
    - nginx
    - curl
    - git
  become: true
```

Ansible runs the task for each item.

B. Loop through a list of dictionaries

```yaml
- name: Create application users
  ansible.builtin.user:
    name: "{{ item.name }}"
    shell: "{{ item.shell }}"
    state: present
  loop:
    - name: deploy
      shell: /bin/bash
    - name: appuser
      shell: /usr/sbin/nologin
  become: true
```

Each iteration uses the current dictionary's values.

C. Loop through a dictionary

```yaml
vars:
  app_ports:
    frontend: 8080
    backend: 9000

tasks:
  - name: Display application ports
    ansible.builtin.debug:
      msg: "{{ item.key }} uses port {{ item.value }}"
    loop: "{{ app_ports | dict2items }}"
```

Output includes:

- `frontend uses port 8080`
- `backend uses port 9000`

Production example: Create multiple users, install packages, configure ports, or deploy several application components using structured data.

Remember: Use `loop` for straightforward iteration. Filters such as `dict2items` help convert data into a structure that is easier to iterate over.

---

99. Custom Modules

Purpose: Extend Ansible with your own module when existing modules do not meet a specific automation requirement.

A custom module is commonly written in Python and accepts parameters, performs an operation, and returns structured results.

Example use case: Your organization has an internal deployment API that no available Ansible module supports.

A simplified custom module might be called `company_deploy.py` and accept an application name and release version.

Example playbook usage:

```yaml
- name: Deploy internal application
  hosts: localhost
  tasks:
    - name: Deploy version
      company_deploy:
        app_name: payment-api
        version: "2.1.0"
      register: deployment_result
```

This assumes `company_deploy` is an actual custom module installed in a location Ansible searches. The example shows how a custom module would be called, not a complete implementation.

Basic custom module responsibilities

1. Define accepted input parameters.
2. Validate the input.
3. Perform the required operation.
4. Report success or failure.
5. Return structured output.
6. Report whether a change occurred.

Production example: Build an internal module that manages a proprietary platform through its API.

Remember: Before writing a custom module, check whether a built-in module, community collection, API module, or existing plugin already solves the problem.

100. Ansible Configuration (`ansible.cfg`)

Purpose: Configure Ansible's default behavior for a project or environment.

The configuration file can define inventory locations, roles paths, connection settings, forks, and other defaults.

Example project structure

```
</> Plain text

ansible-project/
├──── ansible.cfg
├──── inventory.ini
└──── site.yml
```

Example `ansible.cfg`

```
</> INI

[defaults]
inventory = ./inventory.ini
roles_path = ./roles
forks = 10
host_key_checking = True
retry_files_enabled = False
```

Meaning

Setting	Purpose
`inventory`	Default inventory file
`roles_path`	Where Ansible looks for roles
`forks`	Maximum number of parallel worker processes
`host_key_checking`	Verify SSH host keys
`retry_files_enabled`	Controls legacy retry-file generation

Run the playbook

```bash
ansible-playbook site.yml
```

Because the inventory is configured in `ansible.cfg`, you do not need to specify `-i inventory.ini` for this example.

Configuration precedence

Ansible can read settings from several sources. Broadly, the order from lower to higher precedence is:

1. Built-in defaults.
2. Configuration file settings.
3. Environment variables.
4. Command-line options.

Important: Some settings have special precedence rules, so check the documentation for a particular option.

Useful commands:

```bash
# Show the configuration file in use
ansible --version

# Display effective configuration and its sources
ansible-config dump --only-changed

# List configuration options
ansible-config list
```

Production example: Keep project-specific inventory and role paths in version-controlled `ansible.cfg` so team members can run the project consistently.

Security note: Keep SSH host-key checking enabled in production. Disabling it removes an important protection against connecting to an impersonated server.

Quick revision — Level 9

#	Concept	Remember this
84	Blocks	Group related tasks

#	Concept	Remember this
85	Rescue	Handle failures
86	Always	Run cleanup/reporting after a block
87	Error handling	Control failures with `ignore_errors`, `failed_when`
88	Delegation	Execute a task on another host
89	`delegate_to`	Specify the delegated host
90	`run_once`	Run once per batch
91	Local actions	Execute tasks on the control node
92	Serial execution	Process hosts in batches
93	Rolling deployments	Deploy gradually and validate
94	Async tasks	Run long tasks asynchronously
95	Retries / Until	Repeat until a condition is met
96	Wait for	Wait for a port or file condition
97	Conditionals	Control execution with `when`
98	Complex loops	Iterate over structured data
99	Custom modules	Extend Ansible with your own automation
100	`ansible.cfg`	Set Ansible configuration defaults

What should you study first?

Given your goal of understanding Ansible without spending too much time on hands-on labs, use this priority order.

Priority 1 — Must understand

Blocks, rescue, always, `when`, `until`, `wait_for`, `delegate_to`, and `ansible.cfg`.

Priority 2 — Production workflows

`serial`, rolling deployments, `run_once`, delegation, and async tasks.

Priority 3 — Learn when needed

Complex loops, custom modules, and advanced failure recovery patterns.

Final practical exercise

Build one playbook that deploys an application to two web servers:

- [ ] Use `serial: 1` to deploy one server at a time.
- [ ] Use `block` and `rescue` to handle deployment failures.
- [ ] Use `wait_for` to check whether the application port opens.
- [ ] Use `until` to retry an application health check.
- [ ] Use `delegate_to` to run a task on the deployment controller.
- [ ] Use `ansible.cfg` to define your inventory and role paths.

This single exercise covers most of the important Level 9 concepts without requiring separate labs for every topic.